



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

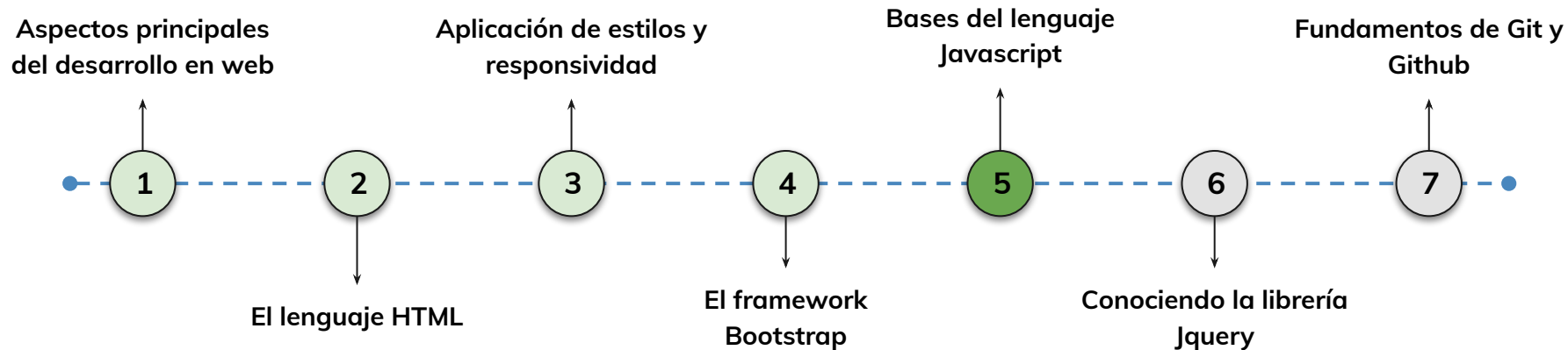


› Bases del lenguaje Javascript - Parte II

AE 5.2: UTILIZAR CÓDIGO JAVASCRIPT PARA LA PERSONALIZACIÓN DE EVENTOS SENCILLOS DENTRO DE UN DOCUMENTO HTML DANDO SOLUCIÓN AL PROBLEMA PLANTEADO.

Hoja de ruta

¿Cuáles skills conforman el programa?



Learning Path

¿Cuáles temas trabajaremos hoy?

13.

Fundamentos de JavaScript II

En esta sesión, profundizaremos en el uso de funciones, manipulación del DOM y eventos en JavaScript, habilidades clave para desarrollar aplicaciones web interactivas y dinámicas.

Funciones y Ámbito de Variables

Tipos de funciones en JavaScript (declaradas, anónimas y flecha).

Ámbito de variables: Diferencias entre var, let y const.

Manipulación del DOM y Eventos

Métodos para acceder y modificar elementos del DOM

Manejo de eventos y prevención de acciones predeterminadas

Objetivos de aprendizaje

¿Qué aprenderás?

- Explorar ejemplos prácticos de funciones y cómo pueden simplificar tu código.
- Comprender la manipulación del DOM con JavaScript.
- Comprender cómo crear eventos interactivos en páginas web utilizando HTML y JavaScript
- Aprender cómo los eventos permiten a los usuarios interactuar con tu sitio web.
- Utilizar la consola de desarrolladores de tu navegador para ejecutar código JavaScript de forma interactiva.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos la importancia de JavaScript en el desarrollo web moderno.
- Exploramos el concepto de variables en JavaScript y cómo se utilizan para almacenar y manipular datos.
- Comprendimos y aplicamos expresiones aritméticas, operadores de asignación y estructuras condicionales (if, if-else) en JavaScript.
- Utilizamos la sentencia switch para realizar múltiples comprobaciones de igualdad de manera eficiente.

Manejo de Javascript

< > Funciones

Cuando se desarrolla una aplicación o sitio web, es muy habitual utilizar una y otra vez las mismas instrucciones.

En programación, una función es un conjunto de instrucciones que se agrupan para realizar una tarea concreta, que luego se puede reutilizar a lo largo de diferentes instancias del código.

Las funciones en JavaScript son bloques de código reutilizables que se pueden invocar para realizar una tarea específica. Proporcionan una forma de encapsular un conjunto de instrucciones y ejecutarlas en cualquier momento que se necesiten. Las funciones pueden aceptar argumentos (parámetros) y devolver un resultado.



< > Funciones

Parámetros

Una función simple, puede no necesitar ningún dato para funcionar. Pero cuando empezamos a codificar funciones más complejas, nos encontramos con la necesidad de recibir cierta información. Cuando enviamos a la función uno o más valores para ser empleados en sus operaciones, estamos hablando de los parámetros de la función.

Los parámetros se envían a la función mediante variables y se colocan entre los paréntesis posteriores al nombre de la función.

```
//ejemplo de sintaxis
function conParametros(parametro1,
parametro2) {
    console.log(parametro1 + " " +
parametro2);
}

//sintaxis de una función con parametros
function saludar(nombre, edad) {
    console.log("¡Hola, " + nombre + "! Tienes
" + edad + " años.");
}

saludar("Juan", 30);

// Muestra "¡Hola, Juan! Tienes 30 años."
```



< > Scope

El scope o ámbito de una variable es la zona del programa en la cual se define, el contexto al que pertenece la misma dentro de un algoritmo, restringiendo su uso y alcance.

JavaScript define dos ámbitos para las variables:

- **Global**
- **Local.**



< > Scope

Ámbito global:

Las variables declaradas fuera de cualquier función tienen un ámbito global. Esto significa que están disponibles en todo el programa y pueden ser accedidas desde cualquier parte. Estas variables se conocen como variables globales y se definen utilizando las palabras clave var, let o const fuera de cualquier función.

```
let nombre = "Juan"; // Variable global

function saludar() {
  console.log("Hola, " + nombre);
  // Accediendo a la variable global dentro de la
  función
}

saludar(); // Imprime "Hola, Juan"
console.log(nombre); // Imprime "Juan"
```



< > Scope

Ámbito local:

Las variables declaradas dentro de una función tienen un ámbito local, lo que significa que solo están disponibles dentro de esa función. Estas variables se conocen como variables locales y se definen utilizando las palabras clave `var`, `let` o `const` dentro de una función.

```
function saludar() {  
  var mensaje = "Hola, bienvenido";  
  // Variable local  
  
  console.log(mensaje);  
}  
  
saludar(); // Imprime "Hola, bienvenido"  
console.log(mensaje);  
// Error: mensaje is not defined
```

✖ ▶ Uncaught ReferenceError: mensaje is not defined



< > Funciones anónimas

Las funciones anónimas son útiles cuando se necesitan funciones temporales o cuando se desea utilizar una función como argumento en otra función, como en el caso de funciones de devolución de llamada (callback functions) o funciones de orden superior (higher-order functions).

Es importante tener en cuenta que, aunque las funciones anónimas no tienen un nombre específico, se asignan a variables u otros elementos para poder ser utilizadas posteriormente.

```
//Generalmente, las funciones anónimas se asignan a variables declaradas como  
constantes  
const suma = function (a, b) { return a + b }  
const resta = function (a, b) { return a - b }  
console.log( suma(15,20) )  
console.log( resta(15,5) )
```



< > Funciones anónimas

Funciones Flechas =>

Se declara una variable `miFuncion` y se le asigna una función flecha como valor. El cuerpo de la función se encuentra dentro de los corchetes {}, donde se pueden escribir las operaciones que realizará la función.

Si la función flecha tiene un solo parámetro, se pueden omitir los paréntesis alrededor del parámetro.

```
const suma = (a, b) => { return a + b }  
//Si es una función de una sola línea con retorno podemos evitar  
escribir el cuerpo.  
const resta = (a, b) => a - b ;  
console.log( suma(15,20) )  
console.log( resta(20,5) )
```



< > Funciones anónimas

Callback

Los callbacks son funciones que se pasan como argumentos a otras funciones. Son una forma común de lograr la ejecución asíncrona y la programación basada en eventos en JavaScript.

La idea principal detrás de los callbacks es permitir que una función sea llamada dentro de otra función después de que se haya completado una tarea o evento específico. Esto permite una programación más flexible y dinámica, ya que puedes definir el comportamiento que se debe ejecutar después de ciertas operaciones.

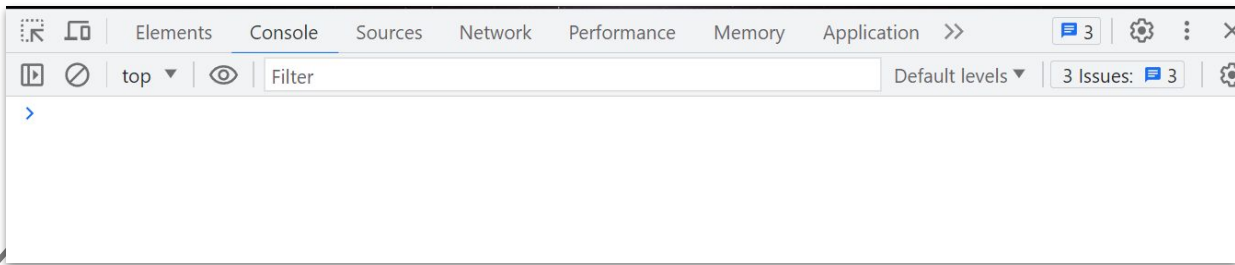
```
function operacionAsincrona(callback) {  
    // Realizar una operación asíncrona  
    // Una vez que la operación se haya  
    // completado, llamar al callback  
    callback();  
}  
  
function miCallback() {  
    console.log("La operación asíncrona se  
    ha completado.");  
}  
  
operacionAsincrona(miCallback);
```



< > **Cómo ejecutar código Javascript en la consola**

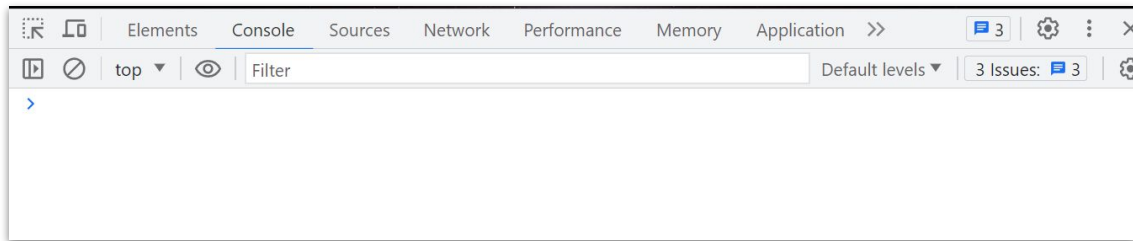
Para ejecutar código JavaScript en la consola del navegador y realizar la depuración del código, puedes seguir estos pasos:

1. Abre cualquier página web en tu navegador.
2. Haz clic derecho en la página y selecciona "Inspeccionar" o presiona la tecla F12 para abrir las herramientas de desarrollo del navegador.
3. En la pestaña "Consola" de las herramientas de desarrollo, verás un prompt donde puedes ingresar y ejecutar código JavaScript. Acá puedes escribir cualquier instrucción o función en JavaScript y presionar "Enter" para ejecutarla.



< > Depurando el código Javascript con la consola

4. Observa los resultados o mensajes de error que se muestran en la consola después de ejecutar el código. puedes utilizar la consola para imprimir valores, realizar cálculos, interactuar con el DOM de la página y depurar problemas en tu código.
5. La consola del navegador es una herramienta muy útil para probar y depurar código JavaScript. Te permite ejecutar código en tiempo real y ver los resultados de tus instrucciones. Además, puedes utilizar métodos específicos de la consola, como `console.log()`, para imprimir valores y mensajes en la consola y facilitar el seguimiento de tu código durante la depuración.

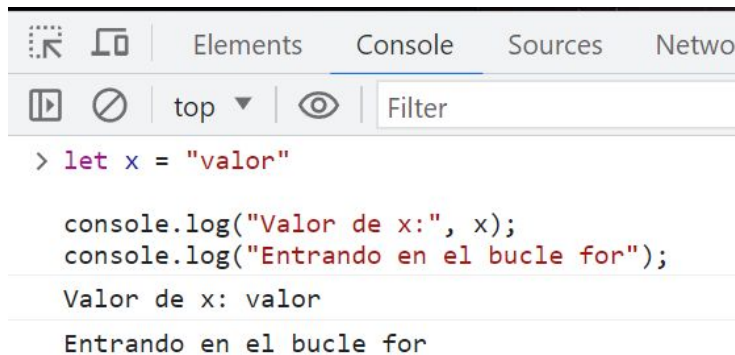


< > Depurando el código Javascript con la consola

La consola del navegador también es una herramienta muy útil para depurar el código JavaScript y encontrar errores o problemas en tu programa. puedes utilizar varias técnicas para depurar y examinar el código en la consola:

1. Mostrar mensajes de depuración:

Puedes utilizar `console.log()` para imprimir mensajes de depuración en la consola. puedes agregar mensajes en diferentes partes de tu código para verificar el valor de variables, comprobar si ciertas condiciones se cumplen o simplemente seguir el flujo de ejecución del programa.



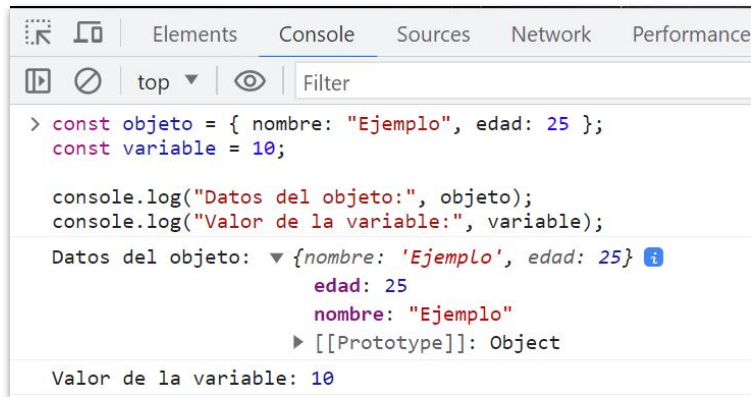
< > Depurando el código Javascript con la consola

2. Inspeccionar objetos y variables:

Puedes utilizar `console.log()` para imprimir el contenido de objetos y variables complejas. Esto te permite examinar su estructura y asegurarte de que contengan los datos esperados.

3. Establecer puntos de interrupción:

Puedes utilizar puntos de interrupción en el código para detener la ejecución en un punto específico y examinar el estado de las variables en ese momento. En las herramientas de desarrollo del navegador, puedes hacer clic en el número de línea a la izquierda del código para establecer un punto de interrupción. Cuando se alcance el punto de interrupción, la ejecución se detendrá y podrás examinar el contexto y las variables en ese punto.



```
> const objeto = { nombre: "Ejemplo", edad: 25 };
const variable = 10;

console.log("Datos del objeto:", objeto);
console.log("Valor de la variable:", variable);

Datos del objeto: {nombre: 'Ejemplo', edad: 25}
  edad: 25
  nombre: "Ejemplo"
  [[Prototype]]: Object

Valor de la variable: 10
```

LIVE CODING

Ejemplo en vivo

¿En qué consistirá la Demo?

En esta sesión en vivo, aprenderemos a utilizar comandos de depuración en JavaScript para identificar y solucionar errores en el código. Usaremos debugger, console.trace(), y console.error() para rastrear problemas y comprender mejor el flujo de ejecución.

1. Preparación:

- Crear un archivo JavaScript (depuracion.js).
- Definir una función con un error intencional:

2. Uso de console.trace()

- Agregar console.trace() para imprimir la pila de llamadas en la consola.

3. Uso de debugger

- Insertar debugger; en un punto clave del código.
- Mostrar cómo pausa la ejecución y permite inspeccionar variables en las herramientas de desarrollo del navegador. Explicar cómo ayuda a rastrear el origen de una ejecución.

4. Uso de console.error()

- Capturar el error de división por cero y mostrarlo con console.error().
- Explicar cómo este comando ayuda a resaltar problemas críticos en la consola.

Tiempo: 15 Minutos

< > **Selectores básicos:** `document.body`

El acceso al DOM se realiza a través del objeto `document`, que es parte del entorno JavaScript en el navegador. El objeto `document` representa el documento HTML cargado y proporciona métodos y propiedades para interactuar con él.

A partir de ahí, podemos utilizar métodos como `getElementById()`, `querySelector()`, `createElement()`, entre otros, para seleccionar, crear y manipular elementos dentro del DOM.

Acceso por objeto `document`

El acceso a `body` usando la referencia `document.body` requiere que el script se incluya al final del `<body>` en el HTML.

```
console.dir(document)
console.dir(document.head)
console.dir(document.body)
```



< > Selectores básicos: `getElementById`

`getElementById()`:

El método `getElementById()` sirve para acceder a un elemento de la estructura HTML, utilizando su atributo ID como identificación.

```
//código HTML
<div id="mi-id">
  <p>Lorem</p>
</div>
//código JS
const miElemento = document.getElementById('mi-id');
```



< > **Selectores básicos:** `getElementsByTagName`

`getElementsByTagName()`:

Este método permite seleccionar elementos por su etiqueta HTML. Recibe como argumento el nombre de la etiqueta y devuelve una colección de elementos que coinciden con esa etiqueta.

```
//código HTML
<div id="mi-id">
  <p>Lorem</p>
</div>
//código JS
const elementosP = document.getElementsByTagName('p');
```



< > Selectores básicos:

getElementsByClassName

getElementsByClassName():

Este método permite seleccionar elementos por su clase CSS. Recibe como argumento el nombre de la clase y devuelve una colección de elementos que tienen esa clase.

```
//código HTML
<div id="mi-id">
  <p class="mi-clase">Este es un párrafo con la clase "mi-clase".</p>
</div>
//código JS
const elementosClase = document.getElementsByClassName('mi-clase');
```



< > Obtención y manipulación de valores y textos de los elementos del DOM

Innet Text:

La propiedad `innerText` de un nodo nos permite modificar su nodo de texto. Es decir, acceder y/o modificar el contenido textual de algún elemento del DOM.

```
const elemento = document.getElementById('mi-elemento');  
elemento.innerText = 'Nuevo texto';
```



< > Obtención y manipulación de valores y textos de los elementos del DOM

Class Name

A través de la propiedad `className` de algún nodo seleccionado podemos acceder al atributo `class` del mismo y definir cuáles van a ser sus clases:

```
const elemento = document.getElementById('mi-elemento');  
elemento.className = 'clase-1 clase-2';
```

Al seleccionar un elemento con el ID utilizando `getElementById()` asignamos un nuevo valor a la propiedad `className` del elemento.

Esto reemplazará todas las clases existentes del elemento por estas nuevas clases.



< > Obtención y manipulación de valores y textos de los elementos del DOM

Class Name

Puedes utilizar `className` para agregar o quitar clases individuales del elemento:

```
const elemento = document.getElementById('mi-elemento');  
elemento.className += ' nueva-clase';
```

Utilizamos el operador `+=` para concatenar la clase "nueva-clase" al atributo class existente del elemento, sin eliminar las clases previas.



< > Agregar o quitar Nodos

createElement()

Para crear un nuevo elemento utilizamos el método `document.createElement()`. Luego, asignamos un contenido de texto al elemento utilizando la propiedad `textContent`. Luego utilizamos el método `appendChild()` para agregarlo al árbol del nodo.

```
// Crear un elemento <p>
const nuevoParrafo = document.createElement('p');
// Agregar contenido al elemento
nuevoParrafo.textContent = 'Este es un nuevo párrafo';

// Obtener el nodo padre al que deseas agregar el nuevo elemento
const contenedor = document.getElementById('mi-contenedor');

// Agregar el nuevo elemento como hijo del nodo padre
contenedor.appendChild(nuevoParrafo);
```



< > Agregar o quitar Nodos

Eliminar elementos

Para quitar nodos del DOM, puedes utilizar el método `removeChild()` para eliminar un nodo específico o utilizar `parentNode.removeChild()` para eliminar un nodo hijo de su nodo padre.

```
// Obtener el nodo que deseas eliminar
const nodoEliminar = document.getElementById('mi-nodo');

// Obtener el nodo padre del nodo a eliminar
const nodoPadre = nodoEliminar.parentNode;

// Eliminar el nodo del DOM
nodoPadre.removeChild(nodoEliminar);
```



< > Agregar o quitar Nodos

remove()

El método `remove()` te permite eliminar un nodo seleccionado del DOM. puedes llamar a este método en el nodo que deseas eliminar y se eliminará del árbol DOM.

```
const elemento = document.getElementById('mi-elemento');  
elemento.remove();
```

Es importante destacar que al utilizar `remove()`, el nodo eliminado no se puede recuperar, por lo que debes tener cuidado al utilizar este método y asegurarte de que realmente deseas eliminar el nodo del DOM.

Hay que tener en cuenta que el método `remove()` no es compatible con todos los navegadores, especialmente versiones antiguas.



< > Obtener datos de Inputs

input

tenemos un formulario con dos campos de entrada:

- uno para el nombre
- otro para el email.

Luego, tenemos un botón que al hacer clic invoca la función `mostrarDatos()`.

```
<form id="mi-formulario">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" />
  <label for="email">Email:</label>
  <input type="email" id="email" />
  <button type="button" onclick="mostrarDatos()">Mostrar Datos</button>
</form>
```



< > Obtener datos de Inputs

input

En la función mostrarDatos(), utilizamos getElementById() para obtener los elementos de entrada del formulario por su ID.

Luego, accedemos a la propiedad value de cada elemento para obtener su valor actual.

```
function mostrarDatos() {  
  // Obtener los valores de los campos de entrada  
  const nombre = document.getElementById('nombre').value;  
  const email = document.getElementById('email').value;  
  
  // Mostrar los datos en la consola o realizar alguna acción con ellos  
  console.log('Nombre:', nombre);  
  console.log('Email:', email);  
}
```



< > Obtener datos de Inputs

querySelector()

Devuelve el primer elemento que coincide con el selector especificado. Si hay varios elementos que coinciden con el selector, solo se seleccionará el primero.

```
<div id="mi-elemento" class="clase1 clase2">
  <p>Contenido del elemento</p>
</div>

// Seleccionar el elemento por su ID
const elementoID = document.querySelector('#mi-elemento');

// Seleccionar el elemento por su clase
const elementoClase = document.querySelector('.clase1');

// Seleccionar el elemento por su etiqueta
const elementoEtiqueta = document.querySelector('p');
```



< > Obtener datos de Inputs

Query Selector All

Query Selector me retorna el primer elemento que coincida con el parámetro de búsqueda, o sea un sólo elemento. Si quiero obtener una colección de elementos puede utilizar el método `querySelectorAll()` siguiendo el mismo comportamiento.

```
<ul>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
</ul>
// Seleccionar todos los elementos <li> dentro del <ul>
const elementos = document.querySelectorAll('ul li');
// Iterar sobre la colección de elementos
elementos.forEach((elemento) => {
  console.log(elemento.textContent);
});
```

LIVE CODING

Ejemplo en vivo

¿En qué consistirá la Demo?

En este live coding, aprenderemos a usar `getElementById` en JavaScript para acceder a elementos del DOM por su ID. Exploraremos cómo obtener y modificar sus valores y textos para interactuar dinámicamente con la página web.

Objetivos:

- Crear una página HTML con al menos tres elementos distintos con IDs únicos (ejemplo: un párrafo, un encabezado y un botón).

Usar JavaScript para:

- Seleccionar un elemento por su ID con `getElementById`.
- Mostrar en la consola su contenido HTML y de texto.
- Modificar su contenido o estilo.
- Reflejar los cambios en la página en tiempo real.

Tiempo: 15 Minutos



Momento:

Time-out!



5 -10 min.



< > Eventos básicos

El método `addEventListener()` es utilizado para asignar un evento a un elemento específico en JavaScript. Permite definir qué evento escuchar y qué función se debe ejecutar como respuesta a ese evento.

```
<button id="mi-boton">Haz clic aquí</button>

const boton = document.getElementById('mi-boton');

boton.addEventListener('click', function() {
  console.log('¡Hola, mundo!');
});
```



< > Eventos básicos

Cuando se hace clic en el botón, se activa el evento click y se muestra un mensaje en la consola. Cuando se realiza un doble clic en el botón, se activa el evento dblclick y se muestra otro mensaje en la consola.

```
<button id="myButton">Haz clic o doble clic aquí</button>

const button = document.getElementById('myButton');
button.addEventListener('click', () => {
  console.log('Has hecho clic en el botón');
});
button.addEventListener('dblclick', () => {
  console.log('Has hecho doble clic en el botón');
});
```



< > Eventos básicos

mousemove: El movimiento del mouse sobre el elemento activa el evento.

click: Se activa después de mousedown o mouseup sobre un elemento válido.

mousedown/mouseup: Se oprime/suelta el botón del ratón sobre un elemento.

mouseover/mouseout: El puntero del mouse se mueve sobre/sale del elemento.

```
//CODIGO HTML DE REFERENCIA
<button id="btnMain">CLICK</button>
//CODIGO JS
let boton = document.getElementById("btnMain")
boton.onclick = () => {console.log("Click")}
boton.onmousemove = () => {console.log("Move")}
```



< > Eventos básicos

keydown: Cuando se presiona.

keyup: Cuando se suelta una tecla.

```
//CODIGO HTML DE REFERENCIA
<input id = "nombre" type="text">
<input id = "edad" type="number">
//CODIGO JS
let input1 = document.getElementById("nombre")
let input2 = document.getElementById("edad")
input1.onkeyup = () => {console.log("keyUp")}
input2.onkeydown = () => {console.log("keyDown")}
```



< > Eventos básicos

Este evento es útil cuando se necesita detectar cambios en el valor de un elemento, como un input de texto o un select, después de que el usuario ha finalizado su interacción con el elemento.

```
<input type="text" id="inputText">

const input = document.getElementById('inputText');

input.addEventListener('change', (event) => {
  const newValue = event.target.value;
  console.log('Nuevo valor:', newValue);
  // Realizar acciones adicionales con el nuevo valor
});
```



< > Eventos básicos

El evento `change` solo se activa cuando hay un cambio real en la selección y se confirma, es decir, cuando se elige una opción diferente. Si el usuario selecciona la misma opción nuevamente, no se activará el evento `change`.

```
<select id="selectOptions">
  <option value="option1">Opción 1</option>
  <option value="option2">Opción 2</option>
  <option value="option3">Opción 3</option>
</select>

const select = document.getElementById('selectOptions');
select.addEventListener('change', (event) => {
  const selectedValue = event.target.value;
  console.log('Valor seleccionado:', selectedValue);
  // Realizar acciones adicionales con el valor seleccionado
});
```

< > Eventos básicos

El evento input es útil cuando se necesita detectar cambios en tiempo real a medida que el usuario escribe o realiza acciones de edición en un campo de texto.

```
<input type="text" id="inputText">

const input = document.getElementById('inputText');

input.addEventListener('input', (event) => {
  const currentValue = event.target.value;
  console.log('Valor actual:', currentValue);
  // Realizar acciones adicionales con el valor actual
});
```



< > Eventos básicos

```
<form id="myForm">
  <label for="name">Nombre:</label>
  <input type="text" id="name" required>
  <button type="submit">Enviar</button>
</form>

const form = document.getElementById('myForm');
form.addEventListener('submit', (event) => {
  event.preventDefault();
  // Evitar el envío del formulario por defecto
  // Obtener los valores de los campos de entrada
  const name = document.getElementById('name').value;
  // Realizar acciones adicionales con los datos ingresados
  console.log('Nombre:', name);
  // Hacer algo más con los datos (enviar a un servidor, procesarlos, etc.)
});
```





Ejercicio N° 1

Función login

Función Login

Contexto: 🙌

Imagina que estás desarrollando un sistema básico de autenticación para una aplicación web. El objetivo es verificar si un usuario puede iniciar sesión con las credenciales correctas antes de acceder al contenido protegido.

Consigna: 📝

Crear una función de inicio de sesión en JavaScript que solicita al usuario un nombre de usuario y una contraseña, verifica si son correctos y devuelve un mensaje de éxito o error.

Tiempo 🕒: 15 minutos

Función Login

Paso a paso: 

Crear la Función de Inicio de Sesión

- Define una función llamada `iniciarSesion` que tome dos parámetros: `usuario` y `contrasena`.
- Dentro de la función, compara el usuario y la contraseña con valores predeterminados para verificar si son correctos. Puedes usar valores de ejemplo como `"usuario123"` y `"claveSecreta"` para este propósito.
- Si el usuario y la contraseña coinciden con los valores predeterminados, la función debe devolver un mensaje de éxito como `"Inicio de sesión exitoso"`.
- Si no coinciden, la función debe devolver un mensaje de error como `"Credenciales incorrectas"`.

Función Login

Paso a paso: 

Solicitar Credenciales al Usuario

- Fuera de la función iniciarSesion, utiliza prompt para solicitar al usuario que ingrese su nombre de usuario y contraseña.
- Almacena los valores ingresados por el usuario en variables.

Llamar a la Función de Inicio de Sesión

- Llama a la función iniciarSesion con los valores ingresados por el usuario como argumentos.
- Almacena el resultado de la función en una variable llamada resultado.

Mostrar el Resultado al Usuario

- Utiliza console.log() para mostrar el valor almacenado en la variable resultado. Esto mostrará un mensaje de éxito o error al usuario.



Ejercicio N° 2

Manipulación del DOM

Manipulación del DOM

Consigna: 📝

Crea una página HTML con al menos **un elemento en el que puedas aplicar la manipulación del DOM** con innerHTML. Puedes usar un párrafo, un div, o cualquier otro elemento de tu elección. Utiliza JavaScript para realizar las siguientes acciones:

- Selecciona el elemento utilizando getElementById u otro selector.
- Utiliza innerHTML para cambiar, agregar o eliminar contenido HTML dentro de ese elemento.
- Actualiza la visualización de la página para reflejar los cambios.

Ejemplos de acciones que puedes realizar:

- Cambiar el texto dentro del elemento.
- Agregar contenido HTML, como listas o imágenes.
- Eliminar contenido HTML del elemento.

Tiempo 🕒: 15 minutos

Manipulación del DOM

Paso a paso: ⚙️

1. En JavaScript, crea una función que se ejecute después de que se cargue el DOM utilizando `document.addEventListener('DOMContentLoaded', function () {...});`.
2. Dentro de la función, utiliza `document.getElementById` u otro selector para seleccionar el elemento que deseas manipular.
3. Utiliza la propiedad `innerHTML` para obtener o establecer el contenido HTML del elemento seleccionado.
4. Puedes asignar nuevo contenido HTML al elemento seleccionado, modificar el contenido existente o eliminarlo.

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Comprendes el concepto de funciones en JavaScript y cómo se utilizan para agrupar y reutilizar bloques de código.**
- ✓ **Has aprendido cómo declarar funciones, incluyendo la especificación de parámetros y el uso de return para devolver valores.**
- ✓ **Manipular el Document Object Model (DOM) de una página web utilizando JavaScript**
- ✓ **Seleccionar elementos HTML en una página utilizando selectores como getElementById y querySelector.**
- ✓ **Comprender cómo crear eventos interactivos en páginas web utilizando HTML y JavaScript.**
- ✓ **Aprender cómo los eventos permiten a los usuarios interactuar con tu sitio web.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

